

1.初次使用者的第一次成功執行GPU運算範例---使用conda

以下提供您 conda + PyTorch 的使用範例：

初始化 conda 然後登出

```
None
$ module load anaconda
$ conda init
$ exit
```

重新登入 (每次登入都要執行module load 不然會沒辦法使用sbatch)

```
None
(base)$ module load slurm anaconda
```

建立 conda 環境並進入該環境

```
None
(base)$ conda create --name my-test-env python=3.11
(base)$ conda activate my-test-env
```

安裝 套件前先修改環境變數，在帳號資料夾下建立tmp資料夾，避免預設tmp塞滿無法安裝程式

```
None
echo 'export TMPDIR=$HOME/tmp' >> ~/.bashrc
mkdir -p ~/tmp
source ~/.bashrc
```

安裝 PyTorch 及所需套件

```
None
(my-test-env)$ conda install pytorch torchvision torchaudio
pytorch-cuda=11.8 -c pytorch -c nvidia
```

Job script 範例

None

```
#!/bin/bash
#SBATCH -J My-Test-Job # Job Name
#SBATCH -o my-test-job.out # Standard output to file
#SBATCH -e my-test-job.err # Standard error to file
#SBATCH --cpus-per-task=4
#SBATCH --mem=32G
#SBATCH --gres=gpu:1
#SBATCH --time=00:30:00
#SBATCH --gres=gpu:1 # Reserve 1 GPUs for this job
#SBATCH -p trialq # Place job into 'trialq' queue

#=====
# Load modules
#=====

module purge
module load anaconda
module list

#=====
# Anaconda Environment
#=====

eval "$(conda shell.bash hook)"

conda deactivate
conda activate my-test-env

#=====
# Execute My Program
#=====

python /home/user/first.py
```

送出 job script

None

```
$ sbatch first.sh
```

2.佇列(partition)的選擇

在將工作派送前,可以先下hpcs 確認現在有哪些GPU是空餘的,可以縮短等待時間。

A:

1. **defq (預設佇列)**

- Job 時間限制: 7 天
- 適合一般計算任務

2. **h200q (高效能佇列)**

- Job 時間限制: 2 天
- 適合需要較高資源但執行時間較短的任務

3. **trialq (測試佇列)**

- Job 時間限制: 30 分鐘
- CPU 限制: 24 cores
- 適合測試或除錯使用

4. 申請獨佔式服務的使用者, 選擇h200q-rsv 作為但JOB時間就沒有限制了

3.是否可以在 HPC 集群上使用 conda 環境

A:

1. Anaconda 應該是可以讓使用者自行安裝使用, 預設是安裝在自己HOME目錄底下, 應該不會影響到系統或其他人

2. 下面是如何載入anaconda環境的範例, 僅供參考。

A@headnode:~\$ module avail

----- /cm/local/modulefiles -----

bcm-post-install/10.0 cm-setup/10.0 gcc/64/4.1.5rc2 null use.own

bcm-superpod-network/10.0 cmd ipmitool/1.8.19 openldap

boost/1.81.0 cmjob luajit python3

cluster-tools/10.0	cmsh	mariadb-libs	python39
cm-bios-tools	dot	module-git	rocm-smi/4.3.0
cm-image/10.0	freeipmi/1.6.10	module-info	shared
cm-scale/cm-scale.module	gcc/13.1.0	modules	slurm/slurm/23.02.4

----- /cm/shared/modulefiles -----

anaconda/2024.02	default-environment	hwloc/1.11.13	openblas/dynamic/0.3.18
cm-pmix3/3.1.4	gdb/13.1	hwloc2/2.8.0	openmpi/gcc/64/4.1.5
cm-pmix4/4.1.1	hdf5_18/1.8.21	mvapich2/gcc/64/2.3.7	openmpi4/gcc/4.1.5
cmake/3.22.6	hpc-x/2.17.1	nvidia-hpc/2024_241	ucx/1.10.1
cmake/3.29.4	hpl/2.3	oneapi/2024.0	

A@headnode:~\$ module load anaconda/2024.02

A@headnode:~\$ conda --version

conda 24.1.2

A@headnode:~\$ conda init

no change	/cm/shared/apps/anaconda/2024.02/condabin/conda
no change	/cm/shared/apps/anaconda/2024.02/bin/conda
no change	/cm/shared/apps/anaconda/2024.02/bin/conda-env
no change	/cm/shared/apps/anaconda/2024.02/bin/activate
no change	/cm/shared/apps/anaconda/2024.02/bin/deactivate
no change	/cm/shared/apps/anaconda/2024.02/etc/profile.d/conda.sh
no change	/cm/shared/apps/anaconda/2024.02/etc/fish/conf.d/conda.fish
no change	/cm/shared/apps/anaconda/2024.02/shell/condabin/Conda.psm1
no change	/cm/shared/apps/anaconda/2024.02/shell/condabin/conda-hook.ps1
no change	
	/cm/shared/apps/anaconda/2024.02/lib/python3.11/site-packages/xontrib/conda.xsh

no change /cm/shared/apps/anaconda/2024.02/etc/profile.d/conda.csh

modified /home/A/.bashrc

==> For changes to take effect, close and re-open your current shell. <==

A@headnode:~\$ cat ~/.bashrc

...

>>> conda initialize >>>

!! Contents within this block are managed by 'conda init' !!

__conda_setup="\$('/cm/shared/apps/anaconda/2024.02/bin/conda' 'shell.bash' 'hook' 2>/dev/null)"

if [\$? -eq 0]; then

 eval "\$__conda_setup"

else

 if [-f "/cm/shared/apps/anaconda/2024.02/etc/profile.d/conda.sh"]; then

 . "/cm/shared/apps/anaconda/2024.02/etc/profile.d/conda.sh"

 else

 export PATH="/cm/shared/apps/anaconda/2024.02/bin:\$PATH"

 fi

fi

unset __conda_setup

<<< conda initialize <<<

4.運行起來會報錯找不到「libnvidia-ml.so.1」

A:

libnvidia-ml.so.1 應該是是NVIDIA GPUDriver的Library, 但headnode沒有GPU, 所以就沒有安裝GPU Driver, 系統上也就不會有**libnvidia-ml.so.1**。

所以用ldd指令查詢gmx_mpi需要連結動態函數庫，就會看到libnvidia-ml.so.1是not found。

dirtyhaostop.bt12@headnode:~\$ ldd opt/gromacs-2024.2/bin/gmx_mpi

```
linux-vdso.so.1 (0x0000155555518000)

libgromacs_mpi.so.9 =>
/home/dirtyhaostop.bt12/opt/gromacs-2024.2/lib/libgromacs_mpi.so.9 (0x000015555046e000)

libmpi.so.40 =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/12.3/hpcx/hpcx-2.17.1/ompi/lib/libmpi.so.40 (0x0000155550000000)

libstdc++.so.6 => /lib/x86_64-linux-gnu/libstdc++.so.6 (0x000015554fc00000)

libgcc_s.so.1 => /lib/x86_64-linux-gnu/libgcc_s.so.1 (0x0000155550439000)

libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x000015554f800000)

libcufftMp.so.11 =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/math_libs/lib64/libcufftMp.so.11
(0x0000155540c00000)

libblas_lp64.so.0 =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/lib/libblas_lp64.so.0
(0x000015553e800000)

libm.so.6 => /lib/x86_64-linux-gnu/libm.so.6 (0x0000155550350000)

libgomp.so.1 =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/lib/libgomp.so.1
(0x000015553d600000)

libmuparser.so.2 => /home/dirtyhaostop.bt12/opt/gromacs-2024.2/lib/libmuparser.so.2
(0x000015554ff7d000)

/lib64/ld-linux-x86-64.so.2 (0x000015555551a000)

libopen-rte.so.40 =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/12.3/hpcx/hpcx-2.17.1/ompi/lib/libopen-rte.so.40 (0x000015553d200000)

libopen-pal.so.40 =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/12.3/hpcx/hpcx-2.17.1/ompi/lib/libopen-pal.so.40 (0x000015553ce00000)

libdl.so.2 => /lib/x86_64-linux-gnu/libdl.so.2 (0x0000155550349000)

librt.so.1 => /lib/x86_64-linux-gnu/librt.so.1 (0x0000155550344000)

libutil.so.1 => /lib/x86_64-linux-gnu/libutil.so.1 (0x000015555033f000)
```

libz.so.1 => /lib/x86_64-linux-gnu/libz.so.1 (0x0000155550321000)

libpthread.so.0 => /lib/x86_64-linux-gnu/libpthread.so.0 (0x000015555031c000)

libnvshmem_host.so.2 =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/nvshmem/lib/libnvshmem_host.so.2 (0x000015553a400000)

libacchost.so =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/lib/libacchost.so
(0x000015553a000000)

libnvcpumath.so =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/lib/libnvcpumath.so
(0x0000155539a00000)

libnvc.so =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/lib/libnvc.so
(0x0000155539600000)

libnvidia-ml.so.1 => not found

GPU Compute Node(DGX H100)是有NVIDIA GPU的, 所以就會安裝GPU Driver, 只要你透過srun進行interactive job到GPU Compute Node裡, 再用ldd指令顯示gmx_mpi需要連結動態函數庫, 就可以看到libnvidia-ml.so.1是連結到系統的/lib/x86_64-linux-gnu/libnvidia-ml.so.1。

dirtyhaostop.bt12@headnode:~\$ srun -p defq --time=00:10:00 --tasks=1 --cpus-per-task=1 --pty /bin/bash -l

dirtyhaostop.bt12@DGX-CN01:~\$ module list

Currently Loaded Modulefiles:

1) slurm/slurm/23.02.4 2) nvidia-hpc/2024_241 3) hpcx 4) nvhpc-hpcx-cuda12/24.1 5) cmake/3.29.4

dirtyhaostop.bt12@DGX-CN01:~\$ ldd opt/gromacs-2024.2/bin/gmx_mpi

linux-ld.so.1 (0x0000155555518000)

libgromacs_mpi.so.9 =>
/home/dirtyhaostop.bt12/opt/gromacs-2024.2/lib/libgromacs_mpi.so.9 (0x000015555046e000)

libmpi.so.40 =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/12.3/hpcx/hpcx-2.17.1/ompi/lib/libmpi.so.40 (0x0000155550000000)

libstdc++.so.6 => /lib/x86_64-linux-gnu/libstdc++.so.6 (0x000015554fdd4000)

libgcc_s.so.1 => /lib/x86_64-linux-gnu/libgcc_s.so.1 (0x0000155550442000)

libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x000015554fbac000)

libcufftMp.so.11 =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/math_libs/lib64/libcufftMp.so.11
(0x0000155541000000)

libblas_lp64.so.0 =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/lib/libblas_lp64.so.0
(0x000015553ec00000)

libm.so.6 => /lib/x86_64-linux-gnu/libm.so.6 (0x0000155550359000)

libgomp.so.1 =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/lib/libgomp.so.1
(0x000015553da00000)

libmuparser.so.2 => /home/dirtyhaostop.bt12/opt/gromacs-2024.2/lib/libmuparser.so.2
(0x000015554fb29000)

/lib64/ld-linux-x86-64.so.2 (0x000015555551a000)

libopen-rte.so.40 =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/12.3/hpcx/hpcx-2.17.1/ompi/lib/libopen-rte.so.40 (0x000015553d600000)

libopen-pal.so.40 =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/12.3/hpcx/hpcx-2.17.1/ompi/lib/libopen-pal.so.40 (0x000015553d200000)

libdl.so.2 => /lib/x86_64-linux-gnu/libdl.so.2 (0x0000155550352000)

librt.so.1 => /lib/x86_64-linux-gnu/librt.so.1 (0x000015555034d000)

libutil.so.1 => /lib/x86_64-linux-gnu/libutil.so.1 (0x0000155550348000)

libz.so.1 => /lib/x86_64-linux-gnu/libz.so.1 (0x000015555032a000)

libpthread.so.0 => /lib/x86_64-linux-gnu/libpthread.so.0 (0x0000155550325000)

libnvshmem_host.so.2 =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/nvshmem/lib/libnvshmem_host.so.2 (0x000015553a85e000)

libacchost.so =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/lib/libacchost.so
(0x000015553a400000)

libnvcpumath.so =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/lib/libnvcpumath.so
(0x0000155539e00000)

libnvc.so =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/lib/libnvc.so
(0x0000155539a00000)

libnvidia-ml.so.1 => /lib/x86_64-linux-gnu/libnvidia-ml.so.1 (0x0000155538a00000)

其實CUDA也有包含**libnvidia-ml.so**，只是檔名少了「.1」，位置就在
[/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/cuda/lib64/stubs](#)目錄，你用echo
\$LD_LIBRARY_PATH，就可以看見有包含到這個目錄，只是gmx_mpi需要連結的是
libnvidia-ml.so.1，所以**libnvidia-ml.so**就被忽略了。

dirtyhaostop.bt12@headnode:~\$ echo \$LD_LIBRARY_PATH

/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/12.3/hpcx/hpcx-2.17.1/mpi/lib:/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/12.3/hpcx/hpcx-2.17.1/ncc
l_rdma_sharp_plugin/lib:/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/12.
3/hpcx/hpcx-2.17.1/sharp/lib:/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/
12.3/hpcx/hpcx-2.17.1/hcoll/lib:/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_li
bs/12.3/hpcx/hpcx-2.17.1/ucc/lib/ucc:/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/co
mm_libs/12.3/hpcx/hpcx-2.17.1/ucc/lib:/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/c
omm_libs/12.3/hpcx/hpcx-2.17.1/ucx/lib/ucx:/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/
24.1/comm_libs/12.3/hpcx/hpcx-2.17.1/ucx/lib:/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_6
4/24.1/comm_libs/nvshmem/lib:/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_li
bs/nccl/lib:/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/math_libs/lib64:/cm/shared/a
pps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/lib:/cm/shared/apps/hpc_sdk/2024_241/Lin
ux_x86_64/24.1/compilers/extras/qd/lib:/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/
cuda/extras/CUPTI/lib64:/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/cuda/lib64:/cm/
shared/apps/slurm/current/lib64/slurm:/cm/shared/apps/slurm/current/lib64:/home/dirtyhaostop.bt
12/opt/gromacs-2024.2/lib:/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/cuda/lib6
4/stubs

dirtyhaostop.bt12@headnode:~\$ ls -alh

[/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/cuda/lib64/stubs/](#)

total 1.9M

drwxr-xr-x 2 root root 4.0K Jan 27 04:11 .

drwxr-xr-x 4 root root 4.0K Jan 27 04:11 ..

-rwxr-xr-x 1 root root 65K Jan 27 04:11 libcuda.so

-rwxr-xr-x 1 root root 5.2K Jan 27 04:11 libnppc.so

-rwxr-xr-x 1 root root 254K Jan 27 04:11 libnppial.so

-rwxr-xr-x 1 root root 134K Jan 27 04:11 libnppicc.so

-rwxr-xr-x 1 root root 174K Jan 27 04:11 libnppidei.so

-rwxr-xr-x 1 root root 258K Jan 27 04:11 libnppif.so

-rwxr-xr-x 1 root root 90K Jan 27 04:11 libnppig.so

-rwxr-xr-x 1 root root 42K Jan 27 04:11 libnppim.so

```
-rwxr-xr-x 1 root root 418K Jan 27 04:11 libnppist.so
-rwxr-xr-x 1 root root 9.2K Jan 27 04:11 libnppisu.so
-rwxr-xr-x 1 root root 54K Jan 27 04:11 libnppitc.so
-rwxr-xr-x 1 root root 218K Jan 27 04:11 libnpps.so
-rwxr-xr-x 1 root root 54K Jan 27 04:11 libnvidia-ml.so
-rwxr-xr-x 1 root root 14K Jan 27 04:11 libnvJitLink.so
-rwxr-xr-x 1 root root 14K Jan 27 04:11 libnvjpeg.so
-rwxr-xr-x 1 root root 5.2K Jan 27 04:11 libnvrtc.so
```

如果你看使用者在編譯gromacs時使用cmake產生的檔案CMakeCache.txt, 可以看到編譯設定其實是用CUDA的libnvidia-ml.so函數庫, 而不是libnvidia-ml.so.1。

```
dirtyhaostop.bt12@headnode:~$ vim src/gromacs-2024.2/build/CMakeCache.txt
```

```
...
```

```
//Path to a library.
```

```
CUDA_nvjpeg_LIBRARY:FILEPATH=/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/cuda/12.3/targets/x86_64-linux/lib/libnvjpeg.so
```

```
//Path to a library.
```

```
CUDA_nvjpeg_static_LIBRARY:FILEPATH=/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/cuda/12.3/targets/x86_64-linux/lib/libnvjpeg_static.a
```

```
//Path to a library.
```

```
CUDA_nvml_LIBRARY:FILEPATH=/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/cuda/12.3/targets/x86_64-linux/lib/stubs/libnvidia-ml.so
```

```
//Path to a library.
```

```
CUDA_nvperf_host_LIBRARY:FILEPATH=/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/cuda/12.3/extras/CUPTI/lib64/libnvperf_host.so
```

```
//Path to a library.
```

```
CUDA_nvperf_host_static_LIBRARY:FILEPATH=/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/cuda/12.3/extras/CUPTI/lib64/libnvperf_host_static.a
```

```
//Path to a library.
```

```
CUDA_nvperf_target_LIBRARY:FILEPATH=/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/cuda/12.3/extras/CUPTI/lib64/libnvperf_target.so
```

```
//Path to a library.
```

```
CUDA_nvptxcompiler_static_LIBRARY:FILEPATH=/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/cuda/12.3/targets/x86_64-linux/lib/libnvptxcompiler_static.a
```

```
...
```

如果你用grep搜尋「libnvidia-ml.so」關鍵字搜尋gromacs cmake編譯的build目錄，會發現編譯時確實是使用CUDA的libnvidia-ml.so，但在測試階段，卻是產生很多錯誤訊息說找不到libnvidia-ml.so.1，所以這可能是其他方面的問題，下面會說明。

```
dirtyhaostop.bt12@headnode:~/src/gromacs-2024.2/build$ grep libnvidia-ml.so * -R
```

```
api/gmxapi/cpp/tests/CMakeFiles/gmxapi-mpi-test.dir/link.txt:/usr/bin/g++ -O3 -DNDEBUG
-Wl,-rpath
-Wl,/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/11.8/hpcx/hpcx-2.14/ompi/lib -Wl,--enable-new-dt
```

```
ags -pthread -Wl,-rpath
-Wl,/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/11.8/hpcx/hpcx-2.14/ompi/lib -Wl,--enable-new-dtags -pthread "CMakeFiles/gmxapi-mpi-test.dir/context.cpp.o"
"CMakeFiles/gm
```

```
xapi-mpi-test.dir/restraint.cpp.o" "CMakeFiles/gmxapi-mpi-test.dir/runner.cpp.o"
"CMakeFiles/gmxapi-mpi-test.dir/status.cpp.o"
"CMakeFiles/gmxapi-mpi-test.dir/stopsignaler.cpp.o" "CMakeFiles/gmxapi-mpi-test.dir/s
```

```
ystem.cpp.o" "CMakeFiles/gmxapi-mpi-test.dir/version.cpp.o"
"CMakeFiles/gmxapi-mpi-test.dir/___/___/___/src/testutils/unittest_main.cpp.o"
../././././src/programs/CMakeFiles/mdrun_objlib.dir/mdrun/mdrun.cpp.o ..
```

```
../././././src/programs/CMakeFiles/mdrun_objlib.dir/mdrun/nonbonded_bench.cpp.o -o
../././././bin/gmxapi-mpi-test
-Wl,-rpath,"$ORIGIN/../lib:/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/cuda/12.3/lib64/st
```

```
ubs" /cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/cuda/12.3/lib64/libcudart_static.a
-ldl /usr/lib/x86_64-linux-gnu/librt.a
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/11.8/hpcx/hpcx-2.14/o
```

mpi/lib/libmpi.so ../../lib/libgmxapi_mpi.so.0.4.0 ../../lib/libmdrun_test_infrastructure.a
../../lib/libtestutils.so ../../lib/libgromacs_mpi.so.9.0.0
/cm/shared/apps/hpc_sdk/2024_241/Lin

ux_x86_64/24.1/cuda/12.3/lib64/stubs/libcuda.so
[/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/cuda/12.3/lib64/stubs/libnvidia-ml.
so](#) -lm /usr/lib/gcc/x86_64-linux-gnu/11/libgomp.so /usr/lib/x86_64-linux-gnu/l

libpthread.a ../../lib/libgmock.so.1.13.0 ../../lib/libgtest.so.1.13.0
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/11.8/hpcx/hpcx-2.14/ompi/lib
libmpi.so -Wl,-rpath-link,/home/dirtyhao

stop.bt12/src/gromacs-2024.2/build/lib

...

tests/essentialdynamics/linfix/make_edi.err:gmh_mpi: error while loading shared libraries:
libnvidia-ml.so.1: cannot open shared object file: No such file or directory

tests/essentialdynamics/linfix/grompp.err:gmh_mpi: error while loading shared libraries:
libnvidia-ml.so.1: cannot open shared object file: No such file or directory

tests/freeenergy/transformAtoB/grompp.err:gmh_mpi: error while loading shared libraries:
libnvidia-ml.so.1: cannot open shared object file: No such file or directory

tests/freeenergy/simtemp/grompp.err:gmh_mpi: error while loading shared libraries:
libnvidia-ml.so.1: cannot open shared object file: No such file or directory

tests/freeenergy/coulandvdwtogether/grompp.err:gmh_mpi: error while loading shared libraries:
libnvidia-ml.so.1: cannot open shared object file: No such file or directory

tests/freeenergy/relative/grompp.err:gmh_mpi: error while loading shared libraries:
libnvidia-ml.so.1: cannot open shared object file: No such file or directory

tests/freeenergy/relative-position-restraints/grompp.err:gmh_mpi: error while loading shared
libraries: **libnvidia-ml.so.1**: cannot open shared object file: No such file or directory

tests/freeenergy/coulandvdwsequential_vdw/grompp.err:gmh_mpi: error while loading shared
libraries: **libnvidia-ml.so.1**: cannot open shared object file: No such file or directory

tests/freeenergy/coulandvdwsequential_coul/grompp.err:gmh_mpi: error while loading shared
libraries: **libnvidia-ml.so.1**: cannot open shared object file: No such file or directory

tests/freeenergy/vdwalone/grompp.err:gmh_mpi: error while loading shared libraries:
libnvidia-ml.so.1: cannot open shared object file: No such file or directory

tests/freeenergy/expanded/grompp.err:gmh_mpi: error while loading shared libraries:
libnvidia-ml.so.1: cannot open shared object file: No such file or directory

tests/freeenergy/restraints/grompp.err:gmh_mpi: error while loading shared libraries:
libnvidia-ml.so.1: cannot open shared object file: No such file or directory

Linux底下有一個指令readelf可以用來查詢執行檔或函數庫記載其內部真正需要連結動態函數庫，ldd只能告訴你要使用執行檔或函數庫所需要的全部動態函數庫，例如執行A需要B函數庫，而B需要C函數庫，C又需要D函數庫，用ldd查詢A，則B、C、D都會算在A需要連結的動態函數庫，而readelf查詢A只會顯示A需要B函數庫，所以用readelf查詢ldd gmx_mpi所顯示的動態函數庫，就可以知道libnvidia-ml.so.1需求連結是從哪裡來的，經過一輪查詢，可以找到libnvidia-ml.so.1連結需求是來自

/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/nvshmem/lib/libnvshmem_host.so.2，這個是來自NVIDIA HPC SDK的函數庫，依此推測NVIDIA HPC SDK預設你是安裝在NVIDIA GPU主機並安裝有GPU Driver的環境。

dirtyhaostop.bt12@headnode:~/opt/gromacs-2024.2/bin\$ ldd gmx_mpi

linux-vdso.so.1 (0x0000155555518000)

libgromacs_mpi.so.9 =>

/home/dirtyhaostop.bt12/opt/gromacs-2024.2/lib/libgromacs_mpi.so.9 (0x000015555046e000)

libmpi.so.40 =>

/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/12.3/hpcx/hpcx-2.17.1/ompi/lib/libmpi.so.40 (0x0000155550000000)

libstdc++.so.6 => /lib/x86_64-linux-gnu/libstdc++.so.6 (0x000015554fc00000)

libgcc_s.so.1 => /lib/x86_64-linux-gnu/libgcc_s.so.1 (0x0000155550439000)

libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x000015554f800000)

libcufftMp.so.11 =>

/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/math_libs/lib64/libcufftMp.so.11 (0x0000155540c00000)

libblas_lp64.so.0 =>

/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/lib/libblas_lp64.so.0 (0x000015553e800000)

libm.so.6 => /lib/x86_64-linux-gnu/libm.so.6 (0x0000155550350000)

libgomp.so.1 =>

/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/lib/libgomp.so.1 (0x000015553d600000)

libmuparser.so.2 => /home/dirtyhaostop.bt12/opt/gromacs-2024.2/lib/libmuparser.so.2

(0x000015554ff7d000)

/lib64/ld-linux-x86-64.so.2 (0x000015555551a000)

libopen-rte.so.40 =>

/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/12.3/hpcx/hpcx-2.17.1/ompi/lib/libopen-rte.so.40 (0x000015553d200000)

libopen-pal.so.40 =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/12.3/hpcx/hpcx-2.17.1/ompi/lib/libopen-pal.so.40 (0x000015553ce00000)

libdl.so.2 => /lib/x86_64-linux-gnu/libdl.so.2 (0x0000155550349000)

librt.so.1 => /lib/x86_64-linux-gnu/librt.so.1 (0x0000155550344000)

libutil.so.1 => /lib/x86_64-linux-gnu/libutil.so.1 (0x000015555033f000)

libz.so.1 => /lib/x86_64-linux-gnu/libz.so.1 (0x0000155550321000)

libpthread.so.0 => /lib/x86_64-linux-gnu/libpthread.so.0 (0x000015555031c000)

libnvshmem_host.so.2 =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/nvshmem/lib/libnvshmem_host.so.2 (0x000015553a400000)

libacchost.so =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/lib/libacchost.so
(0x000015553a000000)

libnvcpumath.so =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/lib/libnvcpumath.so
(0x0000155539a00000)

libnvc.so =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/lib/libnvc.so
(0x0000155539600000)

libnvidia-ml.so.1 => not found

dirtyhaostop.bt12@headnode:~/opt/gromacs-2024.2/bin\$ readelf -d
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/nvshmem/lib/libnvshmem_host.so.2

Dynamic section at offset 0x297c4a0 contains 36 entries:

Tag	Type	Name/Value
0x0000000000000001 (NEEDED)		Shared library: [libnvidia-ml.so.1]
0x0000000000000001 (NEEDED)		Shared library: [libpthread.so.0]
0x0000000000000001 (NEEDED)		Shared library: [libdl.so.2]
0x0000000000000001 (NEEDED)		Shared library: [librt.so.1]
0x0000000000000001 (NEEDED)		Shared library: [libstdc++.so.6]
0x0000000000000001 (NEEDED)		Shared library: [libm.so.6]
0x0000000000000001 (NEEDED)		Shared library: [libgcc_s.so.1]

0x0000000000000001 (NEEDED)	Shared library: [libc.so.6]
0x0000000000000001 (NEEDED)	Shared library: [ld-linux-x86-64.so.2]
0x000000000000000e (SONAME)	Library soname: [libnvshmem_host.so.2]
0x000000000000001d (RUNPATH)	Library runpath: [\${ORIGIN}]
0x000000000000000c (INIT)	0x2c000
0x000000000000000d (FINI)	0x1f0524
0x0000000000000019 (INIT_ARRAY)	0x2979000
0x000000000000001b (INIT_ARRAYSZ)	240 (bytes)

另外，這邊有一個方法，可以將[libnvidia-ml.so](#)做一個超連結到使用者HOME底下gromacs安裝位置的Library目錄(opt/gromacs-2024.2/lib/), 連結名稱改為[libnvidia-ml.so.1](#)，這樣就可以讓gmi_mpi去尋找到，但實際上是否可行，還是要讓使用者測試過才知道。或者使用者也可以透過srun的interactive job到GPU Compute Node進行編譯工作與簡單測試，只不過可能會占用到計算額度。

```
dirtyhaostop.bt12@headnode:~$ ln -s
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/cuda/lib64/stubs/libnvidia-ml.so
opt/gromacs-2024.2/lib/libnvidia-ml.so.1
```

```
dirtyhaostop.bt12@headnode:~$ ldd opt/gromacs-2024.2/bin/gmx_mpi
```

```
linux-vdso.so.1 (0x0000155555518000)

libgromacs_mpi.so.9 =>
/home/dirtyhaostop.bt12/opt/gromacs-2024.2/lib/libgromacs_mpi.so.9 (0x000015555046e000)

libmpi.so.40 =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/12.3/hpcx/hpcx-2.17.1/ompi/lib/libmpi.so.40 (0x0000155550000000)

libstdc++.so.6 => /lib/x86_64-linux-gnu/libstdc++.so.6 (0x000015554fc00000)

libgcc_s.so.1 => /lib/x86_64-linux-gnu/libgcc_s.so.1 (0x0000155550439000)

libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x000015554f800000)

libcufftMp.so.11 =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/math_libs/lib64/libcufftMp.so.11
(0x0000155540c00000)

libblas_lp64.so.0 =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/lib/libblas_lp64.so.0
(0x000015553e800000)
```

libm.so.6 => /lib/x86_64-linux-gnu/libm.so.6 (0x0000155550350000)

libgomp.so.1 =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/lib/libgomp.so.1
(0x000015553d600000)

libmuparser.so.2 => /home/dirtyhaostop.bt12/opt/gromacs-2024.2/lib/libmuparser.so.2
(0x000015554ff7d000)

/lib64/ld-linux-x86-64.so.2 (0x00001555551a000)

libopen-rte.so.40 =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/12.3/hpcx/hpcx-2.17.1/mpi/lib/libopen-rte.so.40 (0x000015553d200000)

libopen-pal.so.40 =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/12.3/hpcx/hpcx-2.17.1/mpi/lib/libopen-pal.so.40 (0x000015553ce00000)

libdl.so.2 => /lib/x86_64-linux-gnu/libdl.so.2 (0x0000155550349000)

librt.so.1 => /lib/x86_64-linux-gnu/librt.so.1 (0x0000155550344000)

libutil.so.1 => /lib/x86_64-linux-gnu/libutil.so.1 (0x000015555033f000)

libz.so.1 => /lib/x86_64-linux-gnu/libz.so.1 (0x0000155550321000)

libpthread.so.0 => /lib/x86_64-linux-gnu/libpthread.so.0 (0x000015555031c000)

libnvshmem_host.so.2 =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/nvshmem/lib/libnvshmem_host.so.2 (0x000015553a400000)

libacchost.so =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/lib/libacchost.so
(0x000015553a000000)

libnvcpumath.so =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/lib/libnvcpumath.so
(0x0000155539a00000)

libnvc.so =>
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/lib/libnvc.so
(0x0000155539600000)

libnvidia-ml.so.1 =>
/home/dirtyhaostop.bt12/opt/gromacs-2024.2/lib/libnvidia-ml.so.1 (0x0000155539200000)

5.我們沒有用過NVIDIA HPC 等等相關的toolkit, 不確定裡面有沒有 CUDA Toolkit 12, 不知道該module load 哪一個會有CUDA Toolkit。

A:如下截圖

Package And Software

Compiler

Compiler	Version	Command	Note
GNU	11.3.0	gcc, gcc-11, g++, g++-11, gfortran, gfortran-11,	BCM-built-in
GNU	12.1.0	gcc-12	BCM-built-in
GNU	13.1.0	gcc, g++, gfortran	BCM-built-in module load gcc/13.1.0
Intel	oneapi-2024.0	ifort, ifx, icc, icx, icpc, icpx	Custom Install module load oneapi/2024.0 module load compiler/2024.0.2
PGI	NVIDIA HPC SDK 24.1	pgf77, pgf90, pgf95, pgfortran, pgcc, pgc++	Custom Install With CUDA 12.3 module load nvidia-hpc/2024_241 module load nvhpc/24.1
NVIDIA	NVIDIA HPC SDK 24.1	nvc, nvcc, nvc++, nvfortran	Custom Install With CUDA 12.3 module load nvidia-hpc/2024_241 module load nvhpc/24.1

The old Intel instructions, ifort, icc, and icpc are now deprecated and will be discontinued late 2024.. The new instructions ifx, icx, and icpx add functionality to transfer computing load to Intel GPUs via OpenMP.

Confidential - For Internal Use Only

KeyWin

Package And Software

NVIDIA HPC-X

Version	Module load	Note
2.17.1	module load hpc-x/2.17.1 module load hpcx	With OpenMPI With GPU Compiler
2.14	module load nvidia-hpc/2024_241 module load nvhpc-hpcx-cuda11/24.1	With CUDA 11.8 OpenMPI With NVIDIA Compiler
2.14	module load nvidia-hpc/2024_241 module load nvhpc-hpcx-cuda12/24.1	With CUDA 12.3 OpenMPI With NVIDIA Compiler

下面是一個例子，教你如何看NVIDIA HPC SDK裡的CUDA 12位置

```
keywinma@headnode:~$ module load nvidia-hpc/2024_241
```

```
keywinma@headnode:~$ module avail
```

```
----- /cm/local/modulefiles -----
```

```
bcm-post-install/10.0    cm-setup/10.0  gcc/64/4.1.5rc2  null          use.own
```

```
bcm-superpod-network/10.0  cmd          ipmitool/1.8.19  openldap
```

```
boost/1.81.0             cmjob        luajit           python3
```

```
cluster-tools/10.0        cmsh         mariadb-libs     python39
```

```
cm-bios-tools            dot          module-git       rocm-smi/4.3.0
```

```
cm-image/10.0            freeipmi/1.6.10  module-info     shared
```

```
cm-scale/cm-scale.module  gcc/13.1.0    modules         slurm/slurm/23.02.4
```

```
----- /cm/shared/modulefiles -----
```

```
cm-pmix3/3.1.4    gdb/13.1      hwloc2/2.8.0      openmpi/gcc/64/4.1.5
```

```
cm-pmix4/4.1.1    hdf5_18/1.8.21  mvapich2/gcc/64/2.3.7  openmpi4/gcc/4.1.5
```

```
cmake/3.22.6      hpc-x/2.17.1   nvidia-hpc/2024_241  ucx/1.10.1
```

```
cmake/3.29.4      hpl/2.3        oneapi/2024.0
```

```
default-environment  hwloc/1.11.13  openblas/dynamic/0.3.18
```

```
----- /cm/shared/apps/hpc_sdk/2024_241/modulefiles -----
```

```
nvhpc-byo-compiler/24.1  nvhpc-hpcx-cuda12/24.1  nvhpc-nompi/24.1      nvhpc/24.1
```

```
nvhpc-hpcx-cuda11/24.1  nvhpc-hpcx/24.1      nvhpc-openmpi3/24.1
```

```
keywinma@headnode:~$ module display nvhpc/24.1
```

```
-----  
/cm/shared/apps/hpc_sdk/2024_241/modulefiles/nvhpc/24.1:
```

```
conflict  nvhpc
```

conflict nvhpc-nompi

conflict nvhpc-byo-compiler

conflict nvhpc-hpcx

conflict nvhpc-openmpi3

setenv NVHPC /cm/shared/apps/hpc_sdk/2024_241

setenv NVHPC_ROOT /cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1

setenv CC
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/bin/nvc

setenv CXX
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/bin/nvc++

setenv FC
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/bin/nvfortran

setenv F90
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/bin/nvfortran

setenv F77
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/bin/nvfortran

setenv CPP cpp

prepend-path PATH
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/cuda/bin

prepend-path PATH
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/bin

prepend-path PATH
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/mpi/bin

prepend-path PATH
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/extras/qd/bin

prepend-path LD_LIBRARY_PATH
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/cuda/lib64

prepend-path LD_LIBRARY_PATH
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/cuda/extras/CUPTI/lib64

prepend-path LD_LIBRARY_PATH
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/extras/qd/lib

prepend-path LD_LIBRARY_PATH
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/lib

```

prepend-path    LD_LIBRARY_PATH
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/math_libs/lib64

prepend-path    LD_LIBRARY_PATH
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/nccl/lib

prepend-path    LD_LIBRARY_PATH
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/nvshmem/lib

prepend-path    CPATH
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/math_libs/include

prepend-path    CPATH
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/nccl/include

prepend-path    CPATH
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/comm_libs/nvshmem/include

prepend-path    CPATH
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/extras/qd/include/qd

prepend-path    MANPATH
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/compilers/man

```

```

-----

keywinma@headnode:~$ ll
/cm/shared/apps/hpc_sdk/2024_241/Linux_x86_64/24.1/cuda

```

```
total 8
```

```
drwxr-xr-x  4 root root  81 Feb 28 01:32 ./
```

```
drwxr-xr-x 10 root root 129 Feb 28 01:32 ../
```

```
drwxr-xr-x 11 root root 4096 Jan 27 04:11 11.8/
```

```
drwxr-xr-x 11 root root 4096 Jan 27 04:11 12.3/
```

```
lrwxrwxrwx  1 root root  8 Feb 28 01:32 bin -> 12.3/bin/
```

```
lrwxrwxrwx  1 root root 12 Feb 28 01:32 include -> 12.3/include/
```

```
lrwxrwxrwx  1 root root 10 Feb 28 01:32 lib64 -> 12.3/lib64/
```

```
lrwxrwxrwx  1 root root  9 Feb 28 01:32 nvvm -> 12.3/nvvm/
```

6.執行時有跑出錯誤

[LOG_CAT_ML] You must specify a valid HCA device by setting:

-x HCOLL_MAIN_IB=<dev_name:port> or -x UCX_NET_DEVICES=<dev_name:port>.

If no device was specified for HCOLL (or the calling library), automatic device detection will be run.

In case of unfounded HCA device please contact your system administrator.

```
[DGX-CN01:2300382] Error: ../../../../ompi/mca/coll/hcoll/coll_hcoll_module.c:310 -  
mca_coll_hcoll_comm_query() Hcol library init failed
```

[LOG_CAT_ML] You must specify a valid HCA device by setting:

-x HCOLL_MAIN_IB=<dev_name:port> or -x UCX_NET_DEVICES=<dev_name:port>.

If no device was specified for HCOLL (or the calling library), automatic device detection will be run.

In case of unfounded HCA device please contact your system administrator.

```
[DGX-CN01:2300381] Error: ../../../../ompi/mca/coll/hcoll/coll_hcoll_module.c:310 -  
mca_coll_hcoll_comm_query() Hcol library init failed
```

A.

因為這次建置並沒有Infiniband Switch, GPU 計算節點(DGX-CN01~02)單單只裝有Infiniband網路卡, 還無法形成Infiniband網路,

如果使用NVIDIA HPC SDK進行編譯程式, 那邊HCOLL ...等這些專為Infiniband網路通信優化的函數庫應該就會包含在裡面,

所以在程式執行時HCOLL等可能就會產生這些訊息, 如果不會造成程式停止計算的話, 其實可以忽略這些信息。

在你的Email沒有提到是怎麼執行, 如果不要看到這些錯誤的話, 可以在執行時試試看下列方式,

```
export OMPI_MCA_coll=^hcoll
```

或

```
mpirun --mca coll ^hcoll ...
```

7.有辦法能讓兩台 node 存取彼此的 GPU嗎

A:

目前兩台之間尚未有高速網路連接, 故無法做跨節點運算

需要等貴單位備齊相關網路設備並完成連接設定後才有環境能做跨節點運算

8.如何確認實驗室帳號被分配到的共享硬碟空間呢？

A:

df -h僅能看到系統上每個mount的總容量與使用容量, quota的部分是設限在儲存上, 也就是你看到掛載於/home的那行, 故無法從您登入的機器上查此資訊.

9.OpenMPI bind to 問題

```
#!/bin/bash
#SBATCH -J tx137215d
#SBATCH -o %x.%j.out
#SBATCH -e %x.%j.err
#SBATCH --nodes=1          # Number of nodes
#SBATCH --ntasks-per-node=5 # Number of MPI tasks per node
#SBATCH --cpus-per-task=2   # Number of cores per MPI task
#SBATCH --gres=gpu:2        # Number of H100 GPU device
#SBATCH --mem=8G            # Maximum memory per node to be allocated
#SBATCH --time=06:00:00
#SBATCH -p defq            # Partition/Queue name
...
# Specify GPU direct communication
export GMX_ENABLE_DIRECT_GPU_COMM=1

export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export NUM_TASKS=$(( ${SLURM_NTASKS_PER_NODE} * ${SLURM_NNODES} ))
...
mpirun -np ${NUM_TASKS} $gmxd mdrun -ntomp ${OMP_NUM_THREADS} -bonded gpu -nb
gpu -pme gpu -pin on -npme 1 -deffnm $curname -c ${curname}.pdb
...
```

檔案「tx137215d.1246.err」報錯節錄:
...

A request was made to bind to that would result in binding more
processes than cpus on a resource:

```
Bind to:  CORE
Node:    DGX-CN01
#processes: 2
#cpus:    1
```

You can override this protection by adding the "overload-allowed"
option to your binding directive.

...

問題一:

這裡不懂的地方是我們以為指定的CPU cores 是`--cpus-per-task=2` 乘以`--ntasks-per-node=5` 等於10這樣, 應該是沒有超過10, 但不懂為什麼 mpirun 有需要overload之類的選項? 而且反過來`--cpus-per-task=5` 乘以`--ntasks-per-node=2` 也是等於10, 但是OpenMPI 就不會有bind to的問題。

另外是我們運行的軟體GROMACS會進行hardware detection, 以下節錄。

檔案「d_10.slurm」指令節錄：

```
...
#!/bin/bash
#SBATCH -J tx137215d
#SBATCH -o %x.%j.out
#SBATCH -e %x.%j.err
#SBATCH --nodes=1          # Number of nodes
#SBATCH --ntasks-per-node=1 # Number of MPI tasks per node
#SBATCH --cpus-per-task=10  # Number of cores per MPI task
#SBATCH --gres=gpu:1        # Request 1 H100 GPU device
#SBATCH --mem=8G            # Maximum memory per node to be allocated
#SBATCH --time=06:00:00
#SBATCH -p defq            # Partition/Queue name
...
export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export NUM_TASKS=$(( ${SLURM_NTASKS_PER_NODE} * ${SLURM_NNODES} ))
...
mpirun -np ${NUM_TASKS} $gmxd run -ntomp ${OMP_NUM_THREADS} -bonded gpu -nb
gpu -pme gpu -pin on -deffnm $curname -c ${curname}.pdb
...
...
```

我們是想指定1個node、10個CPU cores、1個GPU, 根據之前在其它HPC使用經驗, 軟體GROMACS的hardware detect會顯示`Running on 1 node with total 10 cores, 10 processing units, 1 compatible GPU`的紀錄。只是根據檔案「tx137215.d1.log」紀錄節錄：

```
...
Running on 1 node with total 5 cores, 10 processing units, 1 compatible GPU
Hardware detected on host DGX-CN01 (the node of MPI rank 0):
CPU info:
  Vendor: Intel
  Brand: Intel(R) Xeon(R) Platinum 8480C
  Family: 6 Model: 143 Stepping: 8
  Features: aes apic avx avx2 avx512f avx512cd avx512bw avx512vl avx512bf16
avx512secondFMA clflush cmov cx8 cx16 f16c fma htt intel lahf mmx msr nonstop_tsc pcid
pclmuldq pdcmm pdpe1gb popcnt pse rdrnd rdtscp sha sse2 sse3 sse4.1 sse4.2 ssse3 tdt x2apic
  Number of AVX-512 FMA units: 2
Hardware topology: Basic
  Packages, cores, and logical processors:
  [indices refer to OS logical processors]
  Package 0: [ 0 112] [ 1 113]
  Package 1: [ 56 168] [ 57 169] [ 58 170]
  CPU limit set by OS: -1 Recommended max number of threads: 10
GPU info:
  Number of GPUs detected: 1
  #0: NVIDIA NVIDIA H100 80GB HBM3, compute cap.: 9.0, ECC: yes, stat: compatible
...

```

問題二：我們以為用slurm的`--cpus-per-task=10`選項等於指定10個CPU cores, 不懂為什麼軟體偵測只有5個CPU core

A:

問題一:

主要是Job Script裡需求ntasks數目大於GPU資源需求引起的, 每一個task會執行一個mpi process, 並對應跑在一個GPU上, 使用者要求ntasks要求5個(--ntasks-per-node=5), 但GPU只要求2顆(--gres=gpu:2), 5個tasks無法分配在2顆GPU上, 故無法執行, 並產生錯誤訊息「...binding more processes than cpus on a resource...」, 所以當時用者要求ntasks要求2個(--ntasks-per-node=5), 對應GPU要求2顆(--gres=gpu:2), 就能正常執行。

因此, 問題不在於ntasks-per-node * cpus-per-task = 10 cpus, 在於ntasks數目大於需求GPU數目引起的。

另外, 也是因為錯誤訊息沒有完整表達問題所在, 所以造成誤導與困惑。

問題二:

第二個問題, 計算節點在BIOS是有啟用Hyperthread, 在OS系統上也就是Thread(s) per core: 2, 所以你的疑問主要是因這個BIOS設定的影響而產生的。通常在CPU計算節點會將Hyperthread關閉, 但在GPU計算節點則會考慮將這個設定打開,

一個計算節點會有Total CPUs(或叫Threads)等於2 Sockets/Node x 56 Cores per socket x 2 Threads per core = 224 CPUs(Threads),

10.取得資源時是否能自行設定DNS呢？

A:

由於變更系統預設 **DNS** 設定需要 root 權限, 基於安全性考量恕無法開放。
不過您可以試試看在您的程式內忽略系統 DNS, 直接指定要使用的 DNS 伺服器。
以下提供 Python 範例:

```
=====
1. 安裝 dnspython 套件
   pip install dnspython
2. 在 python 程式碼最前面加入以下程式碼
   import dns.resolver
   my_resolver = dns.resolver.Resolver(configure=False)
   my_resolver.nameservers = ['8.8.8.8']
   dns.resolver.override_system_resolver(my_resolver)
=====
```


11.在 HPC 上使用自訂 Docker Image

是否可以使用自訂 Docker image？

可以的，不過我們的系統執行 container 的環境不是 Docker，所以 image 格式需要作轉換處理後才能使用。

具體說明

我們的系統是使用 Slurm 搭配 [enroot](#) 來執行 container，而 enroot 只能使用 SquashFS 檔作為 image，因此您需要先將 Docker image 轉換為 SquashFS 檔 (可以參考[這裡](#)的指令)，然後上傳至 HPC 後再使用 srun 搭配 `--container-image='./my_image.sqsh'` 參數執行。

另外，enroot 會需要 container 裡有 shell (e.g. `/bin/sh`) 才能正常執行，因此若您原本的 container image 中沒有 shell 就會出現錯誤訊息並直接結束，這點與 Docker 不太一樣，Docker 並無強制要求 image 裡要有 shell。

您可以在本地環境先透過以下指令確認 container image 中是不是真的沒有 shell，

```
None  
docker run --rm my-image-name:my-tag /bin/sh
```

如果出現類似 `/bin/sh: no such file or directory` 等訊息則說明沒有 shell。

若確定沒有 shell，則您需要在 build image 時用 docker copy 等方法將 shell 執行檔放到您的 container image 裡再轉換成 SquashFS 檔；或者也可以將轉換好但沒有 shell 的 SquashFS 檔解開、放入 shell 後再重新打包。至於 shell 執行檔可以選用簡單輕量的 busybox 來替代看看。

以下提供您參考步驟，實際使用的指令參數可能仍須根據您的需求調整：

(前置作業) 下載 busybox 並設定可執行權限

```
None  
wget  
https://www.busybox.net/downloads/binaries/1.35.0-x86\_64-linux-musl/busybox
```

```
chmod +x ./busybox
```

(方法 1) 加入 docker copy 重新 build image

在您的 Dockerfile 中加入以下指令,

```
None  
COPY ./busybox /bin/sh
```

然後重新執行您的 docker build 指令, 其餘步驟(e.g. 轉換成 SquashFS、上傳、執行等)則不變。

(方法 2) 重新打包 SquashFS 檔

解開 SquashFS 檔

```
None  
unsquashfs ./my_image.sqsh
```

複製 busybox 並將其作為 shell

```
None  
mkdir -p ./squashfs-root/bin  
cp ./busybox ./squashfs-root/bin/sh
```

重新打包 SquashFS 檔

```
None  
fakeroot mksquashfs ./squashfs-root/ ./my_new_image.sqsh -info -processors 1  
-noappend -comp xz -Xdict-size 100%
```

其餘步驟(e.g. 上傳、執行等)則不變。

12.無法開始運作，狀態始終是PD沒有執行

A:

- 1.務必為每個工作(job)設定正確的資源使用量(包含CPU、Memory、GPU和TimeLimit)，並不得超過申請或規定之上限，否則該工作將可能不會進入可執行狀態而被排程。
- 2.SBATCH 不是指令，而是給排程器看的參數，因此前面的 # 不能拿掉。

13.查看 HPC 使用率

A:

使用方法:輸入 "hpcs" 指令即可

目前只有 headnode 上有該程式，其他 node 無該程式。

14.使用H200 的SBATCH

若使用 H100, 請在腳本中設定:

#SBATCH --partition=defq

若使用 H200, 請在腳本中設定:

#SBATCH --partition=h200q

15.slurm JOB Reason code

A:

使用者在初次碰觸slurm時常常會遇到無法執行任務的狀況，大部分遇到的狀況與無法運作的原因

都可以在下列連結中找到

https://slurm.schedmd.com/job_reason_codes.html

例如當您出現

AssocMaxMemPerJob 時就可以在

job script 內加上 #SBATCH --mem=32G 等設定來解決任務無法執行的問題。

16.請問HPC上有安裝CMake (3.22 以上) 嗎？

A:目前系統已經協助安裝了兩個cmake版本(3.22.6與最新版本3.29.4)

17.PIP無法安裝程式

A:如果是因為共用tmp資料夾已滿無法安裝程式, 可以修改環境變數讓安裝可以順利進行

None

```
echo 'export TMPDIR=$HOME/tmp' >> ~/.bashrc  
mkdir -p ~/tmp  
source ~/.bashrc
```

*.待續